



1. Problem Title & Link

Title: LeetCode 695 – Max Area of Island

Link: <https://leetcode.com/problems/max-area-of-island/>

2. Problem Statement (Short Summary)

You are given an $m \times n$ binary grid where:

- 1 → Land
- 0 → Water

An island is formed by **horizontally or vertically adjacent** land cells.

Return the **maximum area** (number of land cells) among all islands.

If there is no island, return 0.

3. Examples (Input → Output)

Example 1

Input

grid =

```
[
  [0,0,1,0],
  [1,1,1,0],
  [0,1,0,0],
  [1,0,0,1]
]
```

Output

5

Explanation

Island

```
  1
1 1 1
  1
```

Area = 5

Example 2

Input

grid =



```
[
  [0,0,0],
  [0,0,0]
]
```

Output

0

4. Constraints

- $1 \leq m, n \leq 50$
- `grid[i][j]` is either 0 or 1

5. Core Concept (Pattern / Topic)

- Graph Traversal
- DFS
- BFS
- Matrix Traversal
- Connected Components

6. Thought Process (Step-by-Step Explanation)

Observation

This problem is almost identical to **LC 200 – Number of Islands**.

The only difference is:

LC 200

Count islands

LC 695

Find largest island

What is Area?

Area means

Number of connected land cells

Example

```
1 1
```

```
1 0
```

Area

3



Algorithm

Scan every cell.

Whenever an unvisited land cell is found,
perform DFS.

During DFS,
count how many cells belong to this island.

After DFS,

update

$\text{maxArea} = \max(\text{maxArea}, \text{currentArea})$

Continue scanning.

DFS Goal

Unlike LC 200,

DFS does not just mark visited.

It also returns

Area of this island

Why This Works

Each island is explored exactly once.

The DFS count equals the island's area.

Taking the maximum gives the answer.

7. Visual / Intuition Diagram (ASCII)

```
Example
0 0 1 0
1 1 1 0
0 1 0 0
DFS starts here
  *
  ↓
Visits
  ■
  ■ ■ ■
  ■

Area
5
```



8. Pseudocode (Language Independent)

```
maxArea = 0

for every cell

    if land

        area = DFS(cell)

        maxArea = max(maxArea, area)

return maxArea

DFS
if outside grid

    return 0

if water

    return 0

mark visited

return

1 +

up +

down +

left +

right
```

9. Code Implementation

Python (DFS)

```
class Solution:

    def maxAreaOfIsland(self, grid):

        rows = len(grid)
        cols = len(grid[0])

        def dfs(r, c):

            if (
```



```
        r < 0 or
        c < 0 or
        r >= rows or
        c >= cols or
        grid[r][c] == 0
    ):
        return 0

    grid[r][c] = 0

    return (
        1
        + dfs(r + 1, c)
        + dfs(r - 1, c)
        + dfs(r, c + 1)
        + dfs(r, c - 1)
    )

maxArea = 0

for r in range(rows):
    for c in range(cols):

        if grid[r][c] == 1:

            area = dfs(r, c)

            maxArea = max(maxArea, area)

return maxArea
```

Python (BFS)

```
from collections import deque

class Solution:

    def maxAreaOfIsland(self, grid):

        rows = len(grid)
        cols = len(grid[0])

        directions = [(1,0), (-1,0), (0,1), (0,-1)]

        maxArea = 0

        for r in range(rows):
            for c in range(cols):

                if grid[r][c] == 1:
```



```
        queue = deque([(r,c)])

        grid[r][c] = 0

        area = 0

        while queue:

            x, y = queue.popleft()

            area += 1

            for dx, dy in directions:

                nx = x + dx
                ny = y + dy

                if (
                    0 <= nx < rows and
                    0 <= ny < cols and
                    grid[nx][ny] == 1
                ):

                    grid[nx][ny] = 0
                    queue.append((nx, ny))

            maxArea = max(maxArea, area)

    return maxArea
```

Java (DFS)

```
class Solution {

    public int maxAreaOfIsland(int[][] grid) {

        int rows = grid.length;
        int cols = grid[0].length;

        int maxArea = 0;

        for(int r = 0; r < rows; r++){

            for(int c = 0; c < cols; c++){

                if(grid[r][c] == 1){

                    maxArea = Math.max(maxArea, dfs(grid, r, c));

                }

            }

        }

    }

}
```



```
        }
    }

    return maxArea;
}

private int dfs(int[][] grid, int r, int c){

    if(
        r < 0 ||
        c < 0 ||
        r >= grid.length ||
        c >= grid[0].length ||
        grid[r][c] == 0
    )
        return 0;

    grid[r][c] = 0;

    return 1
        + dfs(grid, r + 1, c)
        + dfs(grid, r - 1, c)
        + dfs(grid, r, c + 1)
        + dfs(grid, r, c - 1);
}
}
```

10. Time & Space Complexity

Time Complexity

$O(m \times n)$

Each cell is visited at most once.

Space Complexity

Recursive DFS

$O(m \times n)$

Worst-case recursion stack.

BFS

$O(m \times n)$

Queue may contain the entire island.

11. Common Mistakes / Edge Cases

- Forgetting to return 1 + neighbors.
- Returning 1 instead of accumulated area.
- Not marking visited before recursive calls.



- Traversing diagonal cells.
- Forgetting to update maxArea.

12. Detailed Dry Run

Example

1 1 0

1 0 0

0 1 1

Start

(0,0)

DFS

1

↓

1

↓

1

Area

3

Update

maxArea = 3

Continue scanning.

Second island

1 1

Area

2

Maximum

max(3,2)

=

3



Answer

3

13. Common Use Cases (Real-Life / Interview)

- Image segmentation.
- Flood-fill algorithms.
- Cluster detection.
- Landmass measurement.
- Connected region analysis.

14. Common Traps (Important!)

- Confusing this with LC 200 (counting islands instead of measuring area).
- Forgetting to count the current cell (1 + ...).
- Not marking visited immediately, causing infinite recursion or duplicate counting.
- Considering diagonal neighbors (only up, down, left, and right are valid).

15. Builds To (Related LeetCode Problems)

- LC 200 – Number of Islands
- LC 547 – Number of Provinces
- LC 1971 – Find if Path Exists in Graph
- LC 841 – Keys and Rooms
- LC 994 – Rotting Oranges
- LC 130 – Surrounded Regions
- LC 463 – Island Perimeter

16. Alternate Approaches + Comparison

Approach	Time	Space	Notes
Recursive DFS	$O(m \times n)$	$O(m \times n)$	Most common interview solution
Iterative DFS	$O(m \times n)$	$O(m \times n)$	Avoids recursion depth issues
BFS	$O(m \times n)$	$O(m \times n)$	Equally efficient

17. Why This Solution Works (Short Intuition)

Each island is a connected component of land cells. DFS explores every land cell in one island exactly once while counting them. The returned count is that island's area. By comparing the area of every discovered island, we obtain the maximum area in the grid.



18. Variations / Follow-Up Questions

1. Count the number of islands instead of the largest area (LC 200).
2. Return the coordinates of the largest island.
3. Compute the perimeter of the largest island (LC 463 – Island Perimeter).
4. Find the largest island after changing one water cell to land (LC 827 – Making A Large Island).
5. Consider diagonal connections (8-directional islands).
- 6.